

맵 시스템 (Map System) 기술 명세서

Unity 2D 횡스크롤 액션 플랫폼 - 맵 생성, Parallax 원근감, 이동 플랫폼 물리 및 페이지 제어 아키텍처

01 맵 생성 & 오브젝트 풀링 메커니즘

오브젝트 풀링 (Object Pooling)

MapManager.cs

런타임 프레임 드랍 및 가비지 컬렉션(GC) 부하를 최소화하기 위해 맵 청크(MapChunk)를 파괴하지 않고 재활용합니다.

Pool Structure:

```
Dictionary<GameObject, Queue<GameObject>> mapPools  
Queue<GameObject> activeMaps
```

- ✓ 시작 시 initialChunkPoolSize 만큼 미사용 맵 미리 비활성화 생성
- ✓ 화면 밖으로 지나간 맵 청크는 activeMaps.Dequeue() 후 풀로 반환

셔플백 (ShuffleBag) 무작위 생성

Fair Randomness

동일한 맵 청크가 연속으로 나오는 편향을 방지하고 다양한 맵 구성을 보장합니다.

Palette에서 청크 복사



셔플(무작위 섞기)



Pop하여 맵 배치

- ✓ 주머니(Bag)가 비어있을 때 해당 페이지 Palette 맵들을 채운 후 섞음
- ✓ 테트리스 방식의 균등한 맵 등장 확률 보장

0.5 단위 반올림(SnapToHalfGrid) 스폰 정렬

```
// MapManager.cs - 0.5 grid 정렬 유틸리티
private float SnapToHalfGrid(float value)
{
    return Mathf.Round(value * 2f) / 2f;
}
```

// 다음 맵 위치 계산

```
float nextX = nextSpawnPosition.x + chunkWidth;
nextSpawnPosition = new Vector3(SnapToHalfGrid(nextX), SnapToHalfGrid(nextSpawnPosition.y), 0f);
```

정밀한 타일 겹 방지: 맵 청크가 연속 배치될 때 소수점 오차로 인한 빈틈(Seam)이 생기지 않도록 모든 스폰 좌표를 0.5 Grid 단위로 Snap 정렬합니다.

02 ScriptableObject 기반 데이터 구조

맵 데이터 아키텍처 (Data Architecture)

기획 데이터와 로직을 분리하여 인스펙터에서 손쉽게 맵 팔레트 및 페이지 설정을 조절할 수 있습니다.

클래스명	유형	주요 역할 및 관리 데이터
S0MapData	ScriptableObject	개별 맵 청크의 너비, 높이, 난이도 레벨, 스폰 가중치 저장
S0MapPalette	ScriptableObject	특정 페이지에 사용 가능한 S0MapData 리스트 묶음 관리
S0MapPhaseSpawnSetting	ScriptableObject	페이지별 몬스터/상자 스폰 확률 및 합정 밀도 파라미터 제어
MapChunk.cs	MonoBehaviour	실제 맵 프리팹에 부착되어 소환 위치 (Transform), 바운더리 콜라이더 정보 제공

03 2.5D Parallax 원근감 & 카메라 연출

Parallax 스크립트 기반 원근감

카메라: Orthographic (Z축 = 0 고정)

x 오프셋: $\text{distX} = \text{camMoveDistance.x} * (1 - \text{parallaxEffectX})$

y 오프셋: $\text{distY} = \text{isYTracking} ? \text{camMoveDistance.y} * (1 - \text{parallaxEffectY}) : 0$

- ✓ 0에 가까울수록 원경 (카메라를 그대로 따라감)
- ✓ 1에 가까울수록 근경 (월드 좌표에 고정됨)
- ✓ 1보다 크면 전경(Foreground) 효과 연출

카메라 바운더리 & Blend (화면 튕 방지)

플레이어가 새로운 청크로 진입 시 Cinemachine 바운더리를 부드럽게 전환합니다.

- ✓ `MapChunkBoundaryTrigger`: 플레이어 진입 감지
- ✓ **듀얼 Cinemachine 카메라** 또는 0.5초 Ease In Out Blend를 적용하여 화면 튕(Snap) 완벽 방지

Parallax Y축 제어 코드

```
// Parallax.cs - LateUpdate 물리 이동 연산
private void LateUpdate()
{
    Vector3 camMoveDistance = cam.position - startCamPos;
    bool isYParallaxActive = (mapManager != null
                              && CameraConfinerManager.Instance != null
                              && CameraConfinerManager.Instance.IsYTrackingActive);

    float distX = camMoveDistance.x * (1 - parallaxEffectX);
    float distY = isYParallaxActive ? camMoveDistance.y * (1 - parallaxEffectY) : 0f;

    transform.position = new Vector3(startPos.x + distX, startPos.y + distY, transform.position.z);
}
```

04 이동 플랫폼 물리 & 관성 결합 로직

로컬 좌표계 왕복 & FixedUpdate

맵 스폰 및 이동 시 연동 딜레이를 방지하기 위해 부모 맵 체크 기준 로컬 좌표계 (transform.localPosition)를 사용합니다.

- ✓ 물리 기반 이동: Rigidbody2D.MovePosition() 사용
- ✓ FixedUpdate 타이밍 내 물리 처리로 떨림 현상 제거

플레이어 관성(Inertia) 결합

플레이어가 플랫폼 위에 탑승 시 속도를 합산 처리합니다.

```
OnCollisionEnter2D → Platform Velocity 주입  
PlayerControl.linearVelocity = PlayerX + PlatformX
```

- ✓ 플랫폼 위에서도 자연스럽게 점프 및 이동 가능
- ✓ 씬 뷰 디버깅을 위한 OnDrawGizmos 이동 경로 visualizer 제공

05 페이즈별 난이도 스케일링 & 스폰 규칙

3단계 논리적 페이지 (CurrentLogicalPhase) 진행 구조

페이지	진행 시간	난이도 디자인 중심축	주요 기믹 및 맵 변화
Phase 1	0 ~ 60초	수량적 스케일링 (Introductory)	평지 중심, 가시 함정(SpikeTrap) 및 기본 몬스터 밀도 증가. Y축 카메라 고정.
Phase 2	60 ~ 120초	피지컬 액션 컨트롤 (Mechanical)	복합 이동 플랫폼 패턴(대각/수직), SlowPad 발판 및 관성 컨트롤 필요. Y축 카메라 추적 활성화.
Phase 3	120초 이상	복합 기믹 시너지 (Complex Synergy)	이동 플랫폼 위 공중 몬스터 전투, landing zone 촉수(Tentacle) 스폰, 타 이밍 가시 회피 등.

스폰 금지 영역 (Exclusion Zone Compliance): 상자(Chest)나 몬스터 스폰 시, 플레이어 스폰 지점 주변 마진 및 함정/점프대 바로 위 영역은 스폰 가능 대상에서 엄격히 제외(Exclusion Zone)되어 플레이어의 불합리한 사망을 방지합니다.